Autonomous Robotic Chess Player via LLM-VLA Orchestration

Team Members

Aaron Chris Dsouza (3036440892)
Lem Hui (3035994004)

Abstract

This project implemented an autonomous chess-playing system built around a hierarchical perception, reasoning, and action architecture. The system combined chessboard perception, symbolic board-state validation, chess-engine analysis, LLM-based move selection, simulated camera input, frontend interaction, benchmark evaluation, and robotic manipulation through a Cobot Magic/Piper ROS execution adapter.

The completed system consisted of two integrated subsystems. The first subsystem handled chess perception, board-state logic, LLM orchestration, Stockfish-backed move selection, frontend simulation, logging, and benchmarking. The second subsystem handled robotic-arm execution. The original plan explored robot action models directly, but the final integration path was made more deterministic by converting selected chess moves into calibrated Cobot Magic joint targets rather than relying on a single general prompt-following robot model.

The corrected benchmark showed that the software architecture was functional, but that exact visual move recognition remained the main bottleneck. The best-performing tested vision model, `gpt-5.3-chat`, achieved a 98% successful call rate and 98% legal SAN rate, but only 42% exact move accuracy across 50 rendered 3D chessboard samples. This result showed that producing a legal chess move is much easier than correctly recognising the actual move from the board image.

Repository Implementation

Repository: https://github.com/GalaxyShades/llm-vla-orchestrator

The repository implements the software orchestration layer of the whole system. It contains the backend API, frontend interface, simulated chess-camera input, vision model integration, board-state validation, Stockfish candidate generation, LLM policy selection, persistent game memory, logging, and benchmark scripts.

The backend exposes endpoints for analysing player moves, resetting the game, retrieving game state, saving frontend UI state, and streaming live events. The frontend provides a playable chess interface and a rendered chess-camera view. This allowed the system to simulate a player making a chess move, capture the resulting board image, send it to the backend, validate the observed transition, choose an AI move, and update the game state.

The repository also includes the benchmark harness used to evaluate vision models. The benchmark generated rendered chessboard images, stored ground-truth move labels, ran multiple VLMs, and produced both detailed prediction files and aggregate summary metrics.

The repository therefore provides concrete implementation evidence for the chess perception, reasoning, validation, orchestration, logging, evaluation, and Cobot Magic execution-interface components of the system. The robotic-arm learning experiments were handled as supporting exploration, while the current chess pipeline uses a direct hardware-executor boundary.

Project Idea and Motivation

The project idea was to build an autonomous robotic chess player that could connect high-level reasoning with low-level physical action. Chess was chosen because it is a structured task with clear rules, a known board layout, and objective legal-move constraints. At the same time, physically playing chess is still challenging for a robot because the system must perceive pieces accurately, reason about legal moves, and execute precise pick-and-place actions.

The motivation was to explore how LLMs, VLMs, chess engines, and robotic action models could be combined in a modular system. A chess engine is strong at strategy but cannot see or manipulate a physical board. A vision-language model can interpret images but may make mistakes about exact square locations. A robotic policy can move objects but may not understand chess or recover well from unexpected states. The system therefore separated these responsibilities into different components.

The final project was not simply a chess engine and not simply a robot-control experiment. It was an orchestration project. The central question was how to connect perception, symbolic validation, chess reasoning, LLM decision-making, and physical execution into one usable system.

Project Objective

The objective was to build a robotic chess-playing system capable of completing the full loop from perception to action:

1. observe the chessboard after a player move;
2. infer the player's move from visual input;
3. validate the inferred board transition using chess rules;
4. select an AI response using chess-engine analysis and LLM policy reasoning;
5. translate the selected move into an executable manipulation task;
6. map the selected move to calibrated robot joint targets;
7. physically move the chess piece using the robotic arm;
8. continue the game over multiple turns.

The completed system achieved the software-side orchestration loop and produced a practical interface for Cobot Magic execution. The physical manipulation side was simplified because training robust general robot policies proved difficult within the project timeframe.

Final System Overview

The final system consisted of two major parts: the software orchestration subsystem and the robotic manipulation subsystem.

The software subsystem handled frontend chess interaction, rendered 3D chessboard simulation, simulated camera capture, vision-language model move recognition, FEN-based board-state memory, legal transition validation using `python-chess`, Stockfish candidate move generation, adaptive move selection using an LLM policy agent, move logging, PGN generation, and benchmark artefacts.

This subsystem converted visual board observations into validated symbolic chess states, then selected the system's next move.

The robotic subsystem handled physical chess-piece manipulation. The original direction was to train Pi-zero or ACT based action models for robotic manipulation. In practice, the trained models struggled to reliably learn the task and could get stuck midway. The final integration design therefore used a more deterministic Cobot Magic executor: a selected chess move such as `e2e4` is split into source and destination squares, each square is looked up in a calibrated pose table, and the executor publishes the corresponding 7D joint targets through the Cobot/Piper ROS bridge.

This design still matched the overall architecture. The LLM did not directly control every motor action. Instead, it selected a legal chess move, and deterministic code translated that move into available robot poses. This made the action side more predictable than relying on a single general prompt-following VLA model.

System Architecture

The completed system followed this high-level architecture:

```
Chessboard / simulated or physical observation
            ↓
```

```
Vision-based move recognition
            ↓
Canonical board-state memory
            ↓
Legal transition validation
            ↓
Stockfish move analysis
            ↓
LLM policy agent
            ↓
Selected chess move
            ↓
Cobot Magic executor
            ↓
Calibrated square poses and ROS joint targets
            ↓
Robotic arm executes the move
            ↓
Updated board state and game log
```

The most important design decision was to separate the system into layers. Chess reasoning and robotic manipulation were not handled by a single model. Instead, the chess state was represented symbolically, chess legality was enforced deterministically, and the LLM was used where flexible decision-making or routing was useful.

This modular design made the system more robust than a purely end-to-end approach. The chess engine ensured that candidate moves were legal and strategically meaningful. The LLM policy layer selected between those candidates. The robotic action layer then handled physical movement through calibrated Cobot Magic/Piper joint commands.

Hardware Setup for robotic manipulation

The physical platform was a **Cobot Magic** system in a Mobile ALOHA configuration, equipped with dual **Agilex Piper** arms. Three **Intel RealSense** cameras provided visual input: one mounted on the left wrist, one on the right wrist, and one providing a top-down view of the board. Model training and inference were performed on a high-speed workstation with an **NVIDIA RTX 4090** GPU.

Dataset and Data Collection

The project used two kinds of data: benchmark data for chess vision and calibration data for robotic execution.

The chess vision benchmark data was generated programmatically. Random legal chess positions were created, a legal move was selected, and the resulting after-move position was rendered using the 3D chessboard frontend. Each sample stored the previous FEN, the after-move FEN, the ground-truth move in UCI and SAN notation, the after-piece placement, camera settings, and the rendered image path. This made the dataset reproducible and allowed different VLMs to be compared on the same board states.

The robotic execution data is represented as calibrated board-square poses. Each chess square can be mapped to a 7D joint target `[joint0, joint1, joint2, joint3, joint4, joint5, gripper]`. This made the demo path more deterministic than asking a robot model to infer arbitrary actions directly from language.

Chessboard Perception and Simulation

A major part of the project was the chess vision problem. A physical chess robot needs to understand what move the player made before it can respond. Directly reconstructing a full board state from one image is

difficult, especially when the board is viewed from an angle. Pieces can overlap, the far side of the board can be visually compressed, and similar pieces can be confused.

To make the problem more controlled, the system used the previous accepted board state as memory. Instead of asking the model to infer the entire game from an image alone, the system gave the vision model:

1. the previous board state in FEN format;
2. an image of the board after the player moved.

The vision task was then narrowed to identifying the single legal transition between the previous symbolic state and the after-move image.

The simulated camera was implemented using a rendered 3D chessboard. The frontend used a low-poly 3D chess set, adjustable camera pitch, and adjustable camera distance. This allowed the system to approximate an angled camera view of a physical board while still generating reproducible benchmark data.

This simulation was useful because it created a controlled bridge between pure symbolic chess testing and real-world robotic perception. It allowed the system to test whether multimodal models could recognise moves under perspective distortion without needing to run every experiment on physical hardware.

Specific Model Implementation

Vision model implementation

The vision model was used to recognise the player's move from the previous FEN and the after-move board image. It returned structured JSON containing:

```
{
  "after_piece_placement": "...",
  "move_san": "...",
  "overall_confidence": 0.0
}
```

The system evaluated several vision-language models, including `gpt-5.3-chat`, `gpt-4o`, `Qwen/Qwen3-VL-30B-A3B-Instruct`, and `Qwen/Qwen2.5-VL-72B-Instruct`. The model output was not trusted directly. It was parsed, normalised, and validated against chess rules.

Board-state and validation model

The symbolic board model used FEN as the canonical representation. The previous accepted FEN was stored in game memory, and each predicted after-move board was checked using `python-chess`. This deterministic validation layer prevented illegal or malformed VLM outputs from being accepted as game state.

The validation step checked whether the predicted after-piece placement matched any legal move from the previous board state. It also checked whether the predicted SAN move was legal. If the SAN prediction and board-placement prediction agreed, the system accepted the move with higher confidence. If only one path was legal, the system could use that valid path. If neither was legal, the vision model was retried with feedback.

Chess engine and policy model

Stockfish was used to analyse the board after the player move and generate candidate AI responses. The system did not ask the LLM to invent chess moves. Instead, Stockfish produced legal candidate moves with evaluations, and the LLM policy agent selected one from the candidate list.

This reduced hallucination risk. The chess engine handled legality and move quality, while the LLM handled softer policy decisions such as keeping the game competitive or pedagogically useful.

Robotic action model

The robotic action model side first explored Pi-zero VLA fine-tuning on 200 trajectories covering 20 chess moves. When Pi-zero failed to produce meaningful arm motion—likely due to configuration issues—a diagnostic box-picking task confirmed that ACT could actuate the arm while Pi-zero could not. We then shifted to specialised ACT models (one per move), but the only model trained before abandonment—a7-to-a5 on 60 trajectories—failed in all 10 trials due to piece-level visual confusion and insufficient data.

The final execution plan was more deterministic than direct prompt-based robot control. Instead of expecting one robot model to understand every instruction, the system mapped move instructions to a finite set of calibrated physical locations.

Vision Move Recognition

The use of both `after_piece_placement` and `move_san` was intentional. The SAN move gave a direct interpretation of what the player did. The after-piece placement gave a board-level representation that could be validated against legal transitions.

The backend checked whether:

- the SAN move was legal from the previous FEN;
- the predicted after-piece placement matched a legal successor state;
- the SAN move and after-piece placement agreed with each other.

If both outputs were legal and agreed, the system accepted the move. If only one output was legal, the system could still use the valid path. If neither output was legal, the system retried with feedback telling the model that its previous prediction was not a valid single-move transition.

This design treated the VLM as a perception module, not as a trusted game-state authority. The model could propose a move, but chess legality was checked by deterministic code.

Board-State Logic and Validation

The system used FEN as the canonical board-state representation. This was important because a chess game contains more than piece locations. It also includes turn order, castling rights, en passant availability, halfmove clock, and move number.

The validation layer used `python-chess` to compare the predicted after-move board against every legal move from the previous state. If exactly one legal transition matched the predicted piece placement, the system accepted the move.

This prevented many common VLM errors, such as:

- returning illegal moves;
- moving the wrong colour;
- inventing or deleting pieces;
- returning malformed board states;
- confusing SAN notation;
- producing an after-board that does not correspond to one legal move.

The system also included a fallback assumption that the player intended a legal move. When the observed board state was inconsistent, the system could infer the closest legal transition. This improved robustness, but also created a risk: a wrong vision output could be converted into a plausible but incorrect legal move. This trade-off became clear in the benchmark results, where legal move rates were much higher than exact move accuracy.

Chess Reasoning and Move Selection

After the player move was validated, the system used Stockfish to analyse the resulting position. Stockfish generated multiple candidate responses using MultiPV analysis. Each candidate included:

- UCI move notation;
- SAN move notation;
- engine evaluation;
- centipawn loss compared with the best move.

The system did not simply play the strongest Stockfish move every turn. Instead, it used a difficulty controller and an LLM policy agent to choose from the candidate moves. This was designed to make the system behave more like a chess trainer than a maximum-strength engine.

The difficulty controller estimated player strength using centipawn loss from recent moves. Lower centipawn loss indicated stronger play, while higher centipawn loss indicated weaker play. This estimate was then used to select candidate moves that kept the game competitive.

The LLM policy agent received the shortlisted Stockfish moves and selected one according to the system objective. Importantly, the LLM did not generate arbitrary moves. It could only choose from engine-generated legal candidates. This reduced hallucination risk and kept chess legality under deterministic control.

Robotic Manipulation and Hardware Execution

We first fine-tuned Pi-zero on 200 trajectories covering 20 different chess moves (10 trajectories each). "The Pi-zero training pipeline followed these steps: (1) collect data on the Cobot Magic computer and transfer to the GPU workstation; (2) convert ALOHA data to LeRobot format with task prompts; (3) configure model and training hyperparameters; (4) compute normalisation statistics for the trajectories; and (5) fine-tune with batch size 16 for 60,000 iterations.

The model barely moved the arm, producing only small, ineffective motions. To verify whether this was a configuration issue, we trained both Pi-zero and ACT on a simpler verification task—"pick chess piece and put into the box"—using 60 episodes. ACT completed the task in 2 out of 7 trials, showing it could at least actuate the arm, but failed to recover from wrong moves and did not generalise. Pi-zero again produced only small movements. Because ACT worked while Pi-zero did not on the same data and hardware, we concluded Pi-zero's failure was due to configuration problems rather than insufficient data.

We abandoned Pi-zero and moved to an ACT-based pipeline. However, ACT is less "intelligent" as it does not use a VLM backbone, and cannot take language input. Hence, we originally intended to use five specialised models, each of which performed only one move. However, only one model was trained before the approach was abandoned.

Trained model: a7-to-a5 pawn move. Demonstrations: 60 trajectories. Success rate: 0/10 trials. Major failure modes: The arm reached the board but could not localise the correct starting square or differentiate between visually similar pawns. It frequently got stuck mid-trajectory and failed to complete the grasp-and-move sequence.

Videos showing demos can be found in the OneDrive demo folder.

We estimated that meaningful generalisation would require at least 200 trajectories per model. Collecting 200+ demonstrations for each of five distinct moves was deemed infeasible within the project timeline, so the remaining four models were not trained.

The final execution design therefore used a more deterministic Cobot Magic strategy. The robot side requires a calibrated square-pose table, and the orchestration layer maps selected moves to source and destination poses. In the simplest form, a move such as `e2e4` is executed by moving to the calibrated `e2` pose and then to the calibrated `e4` pose.

The robotic execution flow was:

```
Selected chess move
        ↓
Move split into source and destination squares
        ↓
Cobot executor loads calibrated square poses
        ↓
Robot publishes ROS joint targets
        ↓
Board state is updated or verified
```

This design made the system more realistic under project constraints. Rather than assuming that one model could handle every possible natural-language command, the system decomposed the problem into a finite set of calibrated physical square targets.

Integration Between Chess Reasoning and Robot Action

The integration point between the chess subsystem and the robotic subsystem was the selected chess move. Once the LLM policy agent selected an AI move, the move was represented in UCI notation, such as:

`e2e4`

This could be converted into a physical instruction:

`Move the piece from e2 to e4`

The Cobot executor then selected the calibrated source and destination poses. For example, a move such as `b7b5` could be mapped to the stored `b7` pose followed by the stored `b5` pose.

This separation gave the system a clean interface:

- the chess layer only needed to decide **what move should be made**;
- the robot layer only needed to decide **how to physically perform that move**;
- the Cobot executor connected the two by translating chess squares into robot joint targets.

This modular interface is important because chess reasoning and robotic control are very different problems. Chess requires symbolic legality and strategy. Robot manipulation requires spatial control, grasping, and physical robustness.

Technology Stack

The backend was written in Python. It used `FastAPI` for the web API, `python-chess` for legal move validation and board-state handling, Stockfish for chess analysis, LangChain and OpenAI-compatible clients for LLM and VLM integration, Pydantic for structured validation, and YAML configuration files for system settings.

The frontend was built with React and Vite. It used `chess.js` and `react-chessboard` for the interactive chessboard, and `three.js`, `@react-three/fiber`, and `@react-three/drei` for the rendered 3D chessboard view. Playwright was used for automated benchmark image capture.

The physical platform was a **Cobot Magic** system in a Mobile ALOHA configuration, equipped with dual **Agilex Piper** arms. Three **Intel RealSense** cameras provided visual input: one mounted on the left wrist, one on the right wrist, and one providing a top-down view of the board. Model training and inference were performed on a high-speed workstation with an **NVIDIA RTX 4090** GPU.

Benchmark Methodology

The benchmark evaluated the chess vision component of the whole system. The goal was to test whether multimodal vision models could correctly identify the player's move from a previous FEN and an after-move rendered image.

The benchmark did not evaluate chess playing strength or robotic manipulation. It specifically evaluated visual move recognition, which is the first major step in the full autonomous loop.

Sample generation

The benchmark generated 50 random chess transition samples. For each sample:

1. a random legal game state was generated;
2. one legal move was randomly selected;
3. the resulting after-move position was rendered as a 3D chessboard image;
4. the ground-truth UCI move, SAN move, and after-piece placement were saved.

The rendered images used an angled 3D camera with small pitch and distance variation. This made the benchmark more realistic than a flat 2D board, while still keeping the evaluation reproducible.

Model evaluation

Each model received:

- the previous FEN;
- the rendered after-move image.

Each model returned:

- predicted SAN;
- predicted after-piece placement;
- confidence.

The benchmark measured:

- successful call rate;
- exact move accuracy;
- exact piece placement accuracy;
- legal SAN rate;
- legal after-placement rate;
- runtime for successful calls;
- number of attempts;
- average model confidence.

Retry mechanism

If a model returned an illegal move and illegal board state, the system retried with feedback. This feedback told the model that its previous answer was not a legal single-move transition from the given FEN.

This tested whether the model could recover from invalid predictions after receiving structured correction.

Benchmark Integrity Correction

An earlier benchmark setup was misconfigured. Under specific circumstances, the setup allowed data leakage into the evaluation process. As a result, the benchmark numbers used in the presentation were inaccurate and overestimated system performance.

This issue has since been rectified. The results reported here are from the corrected benchmark output in `data/benchmark_vision/vision_summary.csv`.

This correction is important because the benchmark should measure whether a vision-language model can infer the correct move from the previous FEN and the after-move image. If ground-truth information leaks into the model input or evaluation pathway, the benchmark no longer measures visual recognition ability. The corrected results therefore provide a more realistic assessment of the perception component.

Corrected Benchmark Results

Model	Successful call rate	Exact move accuracy	Exact piece placement accuracy	Legal SAN rate	Legal after-placement rate	Average runtime, successful calls only
`Qwen3-VL-30B-A3B`	10%	0%	0%	10%	10%	3.43 s
`Qwen2.5-VL-72B`	62%	2%	2%	62%	62%	5.33 s
`gpt-5.3-chat`	98%	42%	42%	98%	98%	37.85 s
`gpt-4o`	64%	4%	4%	60%	64%	9.04 s

Notebook figures

The following figures were generated from `benchmark_models.ipynb` using the saved benchmark artefacts.

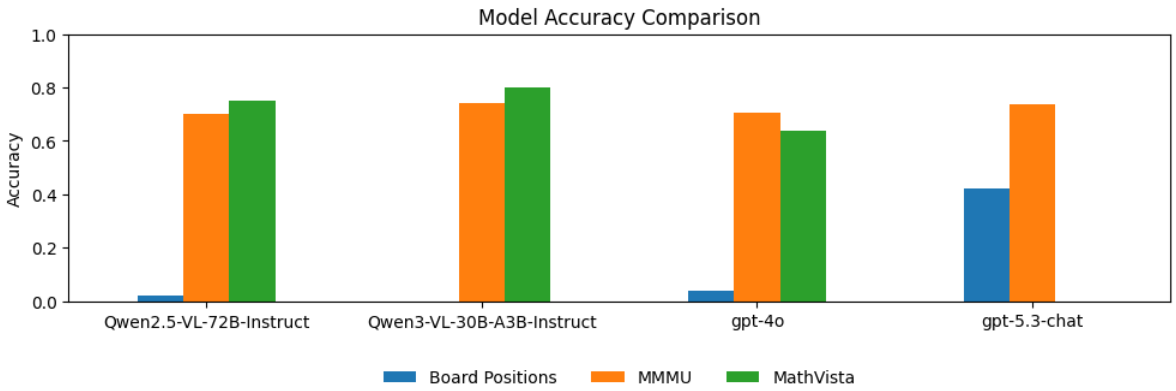

Figure 1: Exact move and exact piece-placement accuracy, shown against external benchmark context from the notebook.

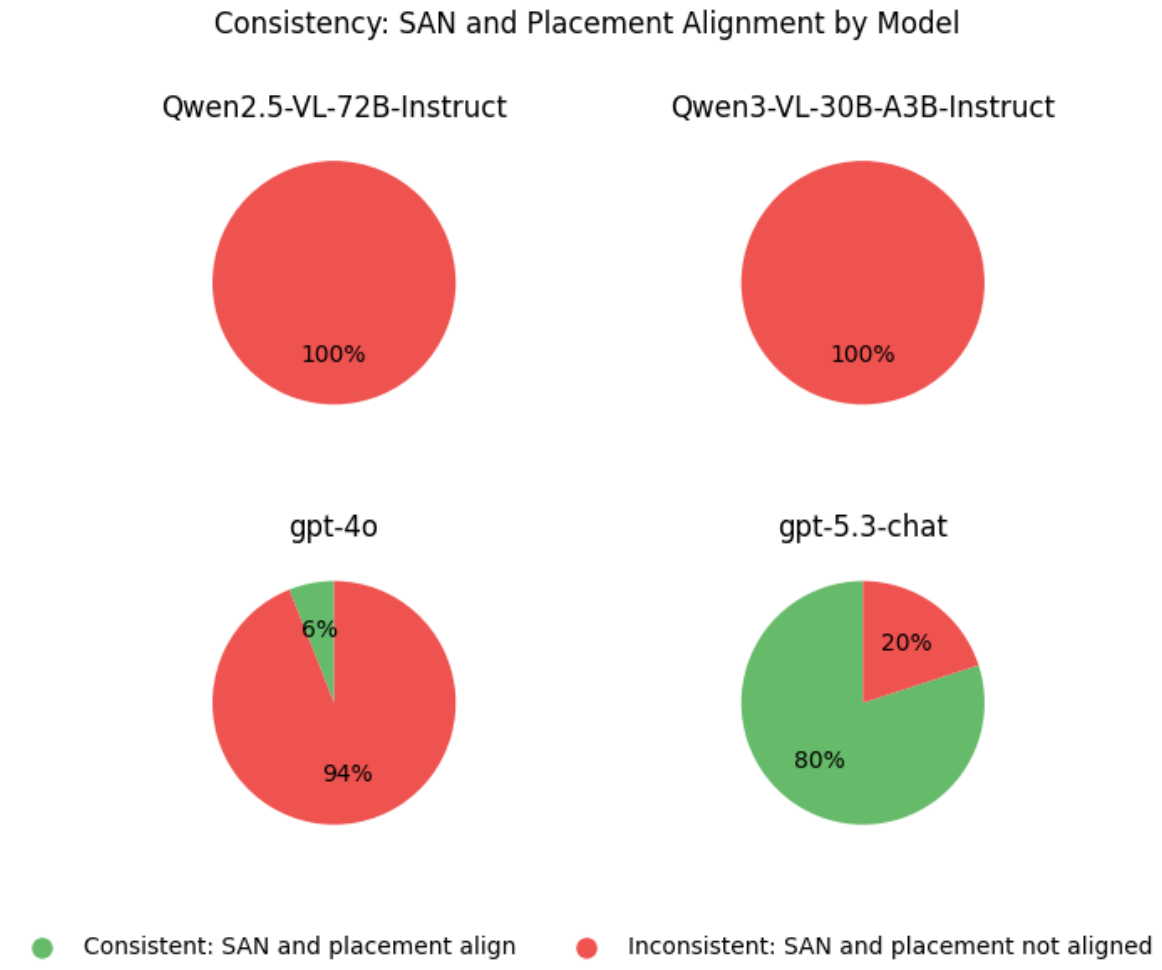

Figure 2: Whether the predicted SAN and predicted after-piece placement agreed with each other.

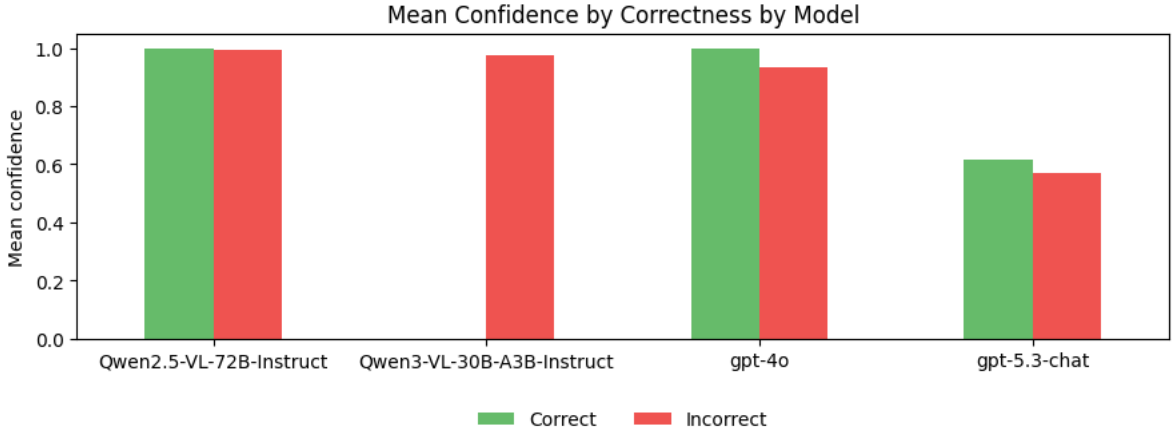

Figure 3: Mean model confidence split by whether the exact move prediction was correct.

Move-selection benchmark diagnostics

The notebook also summarised the move-selection benchmark stored in `data/benchmark_move/benchmark_move_summary.csv`. This benchmark is separate from visual move recognition: it evaluates how each model selected among Stockfish candidate moves once the board state was already known.

Model	Successful runs	Best-candidate pick rate	Average selected CP loss	Forcing check	Material capture	Quiet positional
`gpt-5.3-chat`	50/50	6%	314.48	8%	22%	70%
`gpt-4o`	50/50	12%	170.32	14%	38%	48%
`Qwen2.5-VL-72B`	50/50	4%	257.62	10%	24%	66%
`Qwen3-VL-30B-A3B`	50/50	66%	35.10	26%	48%	26%

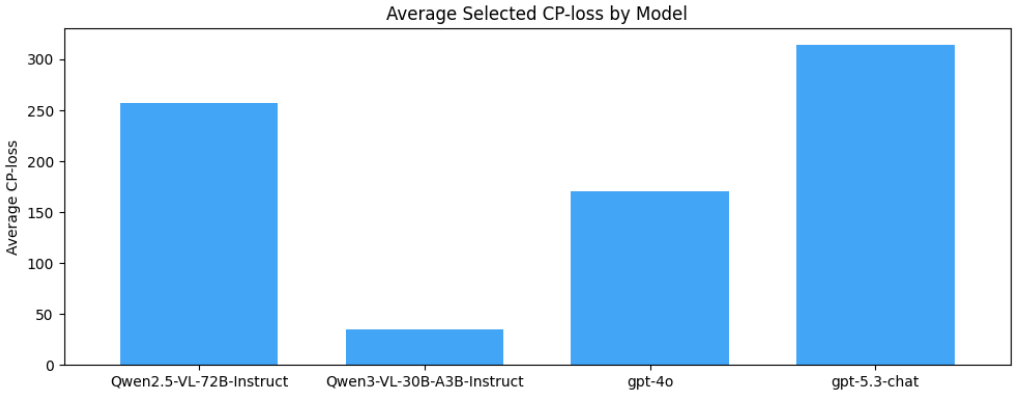

Figure 4: Average selected centipawn loss by model in the move-selection benchmark.

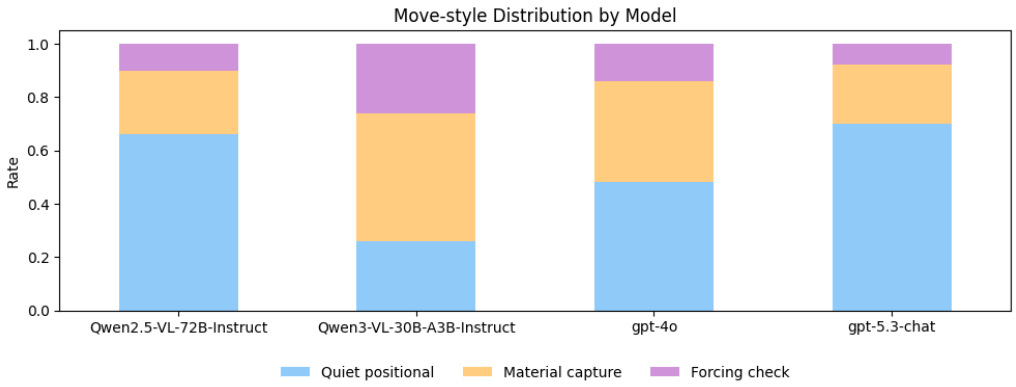

Figure 5: Move-style distribution for successful move-selection benchmark rows.

Results Analysis

Robotic manipulation results

No reliable robotic execution was achieved. Pi-zero was non-functional for chess manipulation despite 200 trajectories, and the diagnostic box-picking task confirmed the issue lay in the Pi-zero configuration rather than data volume. ACT proved capable of actuating the arm on the box task (2/7 success), but its inability to recover from errors indicated poor generalisation and a lack of failure-resolution training data. The specialised ACT approach never produced a working model. The single trained model (a7-to-a5) failed in all 10 trials. The

primary failure modes were piece-level visual confusion—multiple similar-looking pawns prevented the model from grounding to the correct square—and insufficient data for spatial generalisation. Because 60 trajectories was inadequate and scaling to 200+ trajectories per model would be required for five separate behaviours, the robotic subsystem never reached the reliability needed to close the full perception–reasoning–action loop. Consequently, the complete autonomous pipeline was validated only up to the move-selection stage; physical execution remained the unclosed gap of the system.

`gpt-5.3-chat` performed best

`gpt-5.3-chat` was the strongest tested model. It succeeded on 49 out of 50 calls and achieved 42% exact move accuracy. It also achieved 98% legal SAN and legal after-placement rates.

This showed that the model was usually able to produce a legal chess transition. However, it still failed to identify the exact move in more than half of the samples. This is a major limitation for a fully autonomous chess robot because one incorrectly recognised move can corrupt the internal game state.

Legal output was much easier than correct output

The most important finding was the gap between legal output and exact correctness.

For `gpt-5.3-chat`:

- legal SAN rate: 98%;
- exact move accuracy: 42%.

This means that the model often produced a legal move, but not necessarily the move actually shown in the image.

This distinction is critical. In chess, many legal moves may be plausible from a given position. A vision model can use the previous FEN to guess a legal move even if it has not correctly interpreted the image. Legal validation is necessary, but it is not sufficient for reliable perception.

Open-weight VLMs struggled

The tested Qwen models performed poorly on exact move recognition. `Qwen/Qwen2.5-VL-72B-Instruct` achieved 62% successful calls but only 2% exact move accuracy. `Qwen/Qwen3-VL-30B-A3B-Instruct` achieved only 10% successful calls and 0% exact accuracy.

This suggests that chessboard move recognition requires more than general image understanding. It requires precise square-level grounding, piece recognition, perspective interpretation, and comparison between before and after states.

`gpt-4o` was faster but less accurate

`gpt-4o` had a much lower average runtime than `gpt-5.3-chat`, but its exact move accuracy was only 4%. For this system, accuracy was more important than speed because a fast but incorrect perception step would break the game loop.

Confidence was not reliable

Some models reported high confidence even when exact accuracy was low. For example, `Qwen/Qwen2.5-VL-72B-Instruct` had very high average confidence on successful calls but only 2% exact move accuracy.

This means self-reported model confidence should not be treated as a reliable correctness signal. Rule-based validation and ground-truth evaluation are more important.

Key Observations

Chessboard coordinate grounding is difficult

The benchmark showed that VLMs struggle to map pieces to exact chess coordinates in angled 3D images. Even if the model recognises the board and piece types, it may still identify the wrong square.

This is especially difficult when pieces overlap visually or when the perspective compresses the far side of the board.

FEN memory helps but can also encourage guessing

Providing the previous FEN makes the task more constrained. The model only needs to infer one move rather than reconstruct the whole board.

However, this can also encourage plausible guessing. A model may output a legal move from the previous position even when it has not correctly read the image. This explains why legal rates were much higher than exact accuracy.

Deterministic validation is essential

The system would be much less reliable without `python-chess` validation. VLM outputs can be malformed, illegal, or inconsistent. Rule-based validation prevents many invalid states from entering the game memory.

However, deterministic validation cannot solve visual ambiguity. If the model returns the wrong legal move, the chess validator may still accept it.

What Worked

The modular architecture worked well. Separating perception, validation, chess reasoning, LLM policy selection, and robotic execution made the system easier to debug and explain.

The FEN-based memory design also worked well. It reduced the difficulty of the vision problem because the model only needed to infer the transition from a known previous state.

The legal validation layer was effective at catching impossible model outputs. This was important because VLMs can produce plausible-looking but illegal moves.

The benchmark harness worked as an evaluation tool. It generated reproducible test data, compared multiple models, and made it clear that exact move recognition was much harder than legal move generation.

The calibrated-executor idea worked better conceptually than direct prompt-based robot control. Mapping chess instructions to a limited set of physical square poses was more realistic than expecting a single model to handle all manipulation cases.

What Did Not Work

The direct VLM vision approach was not accurate enough for fully reliable autonomous chess play. Even the best model only achieved 42% exact move accuracy.

The Pi-zero VLA approach was non-functional for chess manipulation despite 200 trajectories; the diagnostic box-picking task confirmed the issue lay in Pi-zero configuration rather than data volume. ACT proved capable of actuating the arm on the box task (2/7 success), but its inability to recover from errors indicated poor generalisation and a lack of failure-resolution data. The specialised ACT approach never produced a working model—the single trained a7-to-a5 model failed in all 10 trials, primarily because it could not differentiate between visually similar pawns or localise the correct square. Because 60 trajectories was inadequate and scaling to 200+ trajectories per model for five separate behaviours was infeasible, the robotic subsystem never reached the reliability needed to close the full perception–reasoning–action loop.

The earlier benchmark setup did not work as a valid evaluation because it allowed data leakage under specific circumstances. This was corrected, but it meant that the presentation results were not representative of final system performance.

What Was Surprising

The most surprising result was the large gap between legal move rate and exact move accuracy. `gpt-5.3-chat` could produce legal outputs for almost all samples, but it was correct on fewer than half of them. This showed that chess legality can make model outputs look more reliable than they actually are.

It was also surprising how difficult the robotic manipulation task was, even though moving a chess piece appears simple to a human. The task required consistent perception, precise motor control, good training data, and recovery from small positional errors.

Another important surprise was that a more deterministic fallback, such as calibrated square-pose execution, could be more useful for a demo than a more ambitious but unreliable learned manipulation policy.

Limitations

Vision accuracy remained the main bottleneck

The best exact move accuracy was 42%. This is not sufficient for reliable unsupervised chess play. The system needs better perception before it can be trusted to maintain a correct game state over many turns.

Simulation does not fully match physical camera input

The rendered 3D board approximated real camera input, but it did not include all real-world complications. Physical deployment would add lighting variation, shadows, camera calibration error, imperfect piece placement, hand occlusion, and possible robot-induced board disturbance.

Legal validation can hide perception errors

A legal move is not necessarily the correct move. The benchmark showed that legality-based success metrics can overstate performance if exact move accuracy is not also reported.

VLA pipeline configuration overhead

The long Pi-zero training pipeline introduced significant configuration complexity. The inability to distinguish training bugs from data insufficiency consumed substantial development time and delayed the pivot to alternative approaches.

Robotic failure recovery remained difficult

The ACT-based robotic side could perform some trained actions, but failure recovery was weak. If the robot deviated slightly from the expected state, it could struggle to recover. This is a key limitation for real autonomous manipulation.

Imitation-learning scalability for multi-move robotics

Training specialised ACT models for individual chess moves would have required at least 200 trajectories per model. Collecting, cleaning, and verifying over 1,000 physical demonstrations across five distinct behaviours was infeasible within the project timeline, making the learned-manipulation approach impractical for open-ended chess play.

Improvements Made During the Project

The project changed in several important ways during implementation.

First, the perception problem was narrowed from full board reconstruction to transition recognition using previous FEN plus an after-move image. This made the task more realistic and easier to validate.

Second, the system added deterministic chess-rule validation. This prevented invalid VLM outputs from directly corrupting the game state.

Third, the benchmark setup was corrected after identifying a data leakage issue. This made the final evaluation more grounded, even though the corrected results were less favourable.

Fourth, the robotic execution design evolved through three stages: initial Pi-zero VLA fine-tuning on 200 trajectories, which failed to produce meaningful arm motion; a diagnostic shift to ACT on a box-picking task, which confirmed actuation was possible but revealed poor generalisation; an attempt at specialised ACT models (one per move), which was abandoned after the single trained a7-to-a5 model failed in all 10 trials; and finally a move to calibrated Cobot Magic execution through a hardware adapter. This progression made the action side more deterministic and better matched the practical behaviour of the available robotic execution methods.

Future Improvements

Improve board perception

A stronger perception pipeline would likely require more structure than direct VLM inference. Possible improvements include:

- detecting board corners;
- estimating a homography;
- dividing the board into 64 squares;
- detecting pieces square by square;
- using a trained chess-piece detector;
- comparing before and after images directly;
- combining classical computer vision with VLM reasoning.

Use multiple camera views

A single angled camera can cause occlusion. Multiple camera views could make piece localisation more reliable.

Add uncertainty-aware confirmation

If the vision model returns a low-confidence move, or if the SAN and after-piece placement disagree, the system could request another image, use a different model, or ask for manual confirmation.

Expand benchmark coverage

The benchmark could be expanded to include:

- captures;
- castling;
- promotions;
- checks;
- crowded positions;
- edge moves;
- visually ambiguous positions;
- real physical board images.

This would provide a more complete picture of system reliability.

Strengthen robot execution recovery

The robotic subsystem would benefit from explicit recovery behaviours. For example, if a grasp fails or a piece is displaced, the robot should be able to re-observe the board, re-localise the target, and retry.

Conclusion

The project implemented a complete autonomous chess system architecture connecting perception, symbolic validation, chess reasoning, LLM policy selection, and robotic execution. The software subsystem handled board-state tracking, vision-based move recognition, Stockfish-backed reasoning, LLM move selection, frontend simulation, logging, and benchmarking.

The robotic subsystem explored both Pi-zero VLA fine-tuning and specialised ACT models before settling on a deterministic Cobot Magic executor. Pi-zero failed to produce functional arm motion despite 200 trajectories, and the only trained ACT model (a7-to-a5) failed in all 10 trials. The calibrated-executor design was therefore adopted as a necessary fallback when learned manipulation proved unreliable within the available timeline.

The corrected benchmark showed that perception remained the main bottleneck. `gpt-5.3-chat` achieved strong legal output rates but only 42% exact move accuracy, showing that producing a legal chess move is much easier than correctly recognising the actual move from an image.

The final system demonstrated the value of modular design. Chess legality, engine analysis, LLM policy reasoning, and robotic manipulation were separated into distinct layers. This made the system easier to debug, evaluate, and adapt when the original VLA execution plan proved less practical than calibrated hardware execution.

Overall, the project showed that an autonomous robotic chess player is feasible as a layered system, but reliable physical autonomy depends on improving exact board perception and robotic failure recovery.